

Rajalakshmi Engineering College

Name: Mahathi B

Email: mahathi.b.2024.csd@rajalakshmi.edu.in

Roll no: 241701029

Phone: 9841411040

Branch: REC

Department: CSD

Batch: 2028

Degree: B.E - CSD

Scan to verify results



2024_28_IV_CSD_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 10_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : COD

1. Problem Statement

Arjun is working on a program that checks if one set of numbers is a subset of another. If Set B is a subset of Set A, the program should print "YES" followed by the sorted elements of Set B. If Set B is not a subset of Set A, the program should print "NO" followed by the average of all elements from both sets combined, rounded to two decimal places.

Implement a class Solution with the required method to perform the subset check using TreeSet in Java.

Input Format

The first line contains an integer n - the number of elements in Set A.

The second line contains n space-separated integers - the elements of Set A.

The third line contains an integer m - the number of elements in Set B.

The fourth line contains m space-separated integers - the elements of Set B.

Output Format

If Set B is a subset of Set A, print "YES" followed by the sorted values of Set B.

Otherwise, print "NO" followed by the average of all numbers in both sets (rounded to two decimal places).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

1 2 3 4 5

3

2 3 5

Output: YES 2 3 5

Answer

```
import java.util.*;
```

```
// You are using Java
```

```
class Solution {
```

```
    public static void checkSubset(TreeSet<Integer> setA, TreeSet<Integer> setB,  
    int totalCount, long sum) {
```

```
        if (setA.containsAll(setB)) {  
            System.out.print("YES ");  
            for (int x : setB) {  
                System.out.print(x + " ");  
            }  
        }
```

```
    } else {  
        System.out.print("NO ");
```

```
        double avg = (double) sum / totalCount;  
        System.out.printf("%.2f", avg);
```

```

    }
}

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        TreeSet<Integer> setA = new TreeSet<>();
        long sum = 0;
        for (int i = 0; i < n; i++) {
            int num = sc.nextInt();
            setA.add(num);
            sum += num;
        }
        int m = sc.nextInt();
        TreeSet<Integer> setB = new TreeSet<>();
        for (int i = 0; i < m; i++) {
            int num = sc.nextInt();
            setB.add(num);
            sum += num;
        }
        Solution.checkSubset(setA, setB, n + m, sum);
        sc.close();
    }
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

David is managing an employee database where each employee has a unique ID, name, and department. He wants to ensure that duplicate employee IDs are not added to the system. Implement a Java program that allows adding employees to the system, displaying all employees, and checking if an employee exists based on the given ID.

Implement a class `EmployeeDatabase` that contains a `HashSet` to store employee records. The `Employee` class should be a user-defined object containing employee details. The main class should handle user operations and interact with the `EmployeeDatabase` class.

Input Format

The first line contains an integer n representing the number of employees to be added.

The next n lines follow, each containing:

1. An integer `employee_id`
2. A string `name`
3. A string `department`

The next line contains an integer m representing the number of queries.

The next m lines follow, each containing an employee ID to check for existence.

Output Format

The output prints a list of all employees added in the format:

"ID: <employee_id>, Name: <name>, Department: <department>"

For each query, output "Employee exists" if the ID is found, otherwise "Employee not found".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

101 John IT

102 Alice HR

103 Bob Finance

2

101

104

Output: ID: 101, Name: John, Department: IT

ID: 102, Name: Alice, Department: HR

ID: 103, Name: Bob, Department: Finance

Employee exists

Employee not found

Answer

```
import java.util.*;

// You are using Java
class Employee {
    int id;
    String name;
    String department;

    Employee(int id, String name, String department) {
        this.id = id;
        this.name = name;
        this.department = department;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Employee)) return false;
        Employee e = (Employee) obj;
        return this.id == e.id;
    }

    @Override
    public int hashCode() {
        return Integer.hashCode(id);
    }
}

class EmployeeDatabase {
    HashSet<Employee> set = new HashSet<>();

    // FIX 1: method signature changed
    void addEmployee(int id, String name, String department) {
        set.add(new Employee(id, name, department));
    }

    // Display all employees
    void displayEmployees() {
        for (Employee e : set) {
            System.out.println("ID: " + e.id + ", Name: " + e.name + ", Department: " +
                e.department);
        }
    }
}
```

```

    }
}

// FIX 2: return boolean
boolean checkEmployee(int id) {
    for (Employee e : set) {
        if (e.id == id) {
            return true;
        }
    }
    return false;
}
}

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        EmployeeDatabase db = new EmployeeDatabase();
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            String name = sc.next();
            String department = sc.next();
            db.addEmployee(id, name, department);
        }
        db.displayEmployees();
        int m = sc.nextInt();
        for (int i = 0; i < m; i++) {
            int id = sc.nextInt();
            if (db.checkEmployee(id))
                System.out.println("Employee exists");
            else
                System.out.println("Employee not found");
        }
        sc.close();
    }
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Bob wants to develop a score-tracking application for a gaming tournament. Each player's score is stored in a HashMap with the player's name as the key and the score as the value.

Write a program to assist Bob that takes user input to enter player scores, calculates the maximum score from the HashMap, and prints the player with the highest score.

Input Format

The input consists of strings representing player details in the format "playerName:score".

The input is terminated by entering "done".

Output Format

The output displays a string, representing the player's name who scored the maximum.

If the value is not numeric, print "Invalid input".

If any special characters other than ':' are given, print "Invalid format".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Alice:15

Bob:56

done

Output: Bob

Answer

```
import java.util.*;
```

```
// You are using Java
```

```
class ScoreTracker {
```

```
    HashMap<String, Integer> scoreMap = new HashMap<>();
```

```

// Process each input line
boolean processInput(String input) {

    // Check format
    if (!input.contains(":") || input.contains("-")) {
        System.out.println("Invalid format");
        return false;
    }

    String[] parts = input.split(":");

    if (parts.length != 2) {
        System.out.println("Invalid format");
        return false;
    }

    String name = parts[0];
    String value = parts[1];

    try {
        int score = Integer.parseInt(value);
        scoreMap.put(name, score);
    } catch (NumberFormatException e) {
        System.out.println("Invalid input");
        return false;
    }

    return true;
}

// Find player with maximum score
String findTopPlayer() {
    int max = Integer.MIN_VALUE;
    String result = "";

    for (Map.Entry<String, Integer> entry : scoreMap.entrySet()) {
        if (entry.getValue() > max) {
            max = entry.getValue();
            result = entry.getKey();
        }
    }
}

```



```

        return result;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ScoreTracker tracker = new ScoreTracker();
        boolean validInput = true;

        while (true) {
            String input = scanner.nextLine();

            if (input.toLowerCase().equals("done")) {
                break;
            }

            if (!tracker.processInput(input)) {
                validInput = false;
                break;
            }
        }

        if (validInput && !tracker.scoreMap.isEmpty()) {
            System.out.println(tracker.findTopPlayer());
        }

        scanner.close();
    }
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

The city library maintains a record of books available for lending. Each book is uniquely identified by its ISBN number, along with its title and author. The librarian wants to efficiently store and manage these records, ensuring books can be listed in the order they were added.

Your task is to implement a Library Management System using HashSet where:

The librarian adds books with ISBN, title, and author. The librarian can remove books by providing an ISBN. Finally, the librarian displays the available books in the order they were added.

Implement a class Library that will handle these operations. The main function should manage user input and interact with the Library class accordingly.

Input Format

The first line contains an integer n – the number of books to be added.

The next n lines contain three values: ISBN (integer), Title (string without spaces), and Author (string without spaces).

1. An integer employee_id
2. A string title
3. A string author name

The next line contains an integer m – the number of books to be removed.

The next m lines follow, each contains an ISBN number to remove.

Output Format

The output prints a list of books available in the library after performing all operations in the format:

"ISBN: <isbn>, Title: <title>, Author: <author>"

If no books remain, print: "No books available"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

1234 JavaCompleteGuide JohnDoe

5678 PythonBasics JaneDoe
9012 DataStructures AliceSmith
1

5679

Output: ISBN: 1234, Title: JavaCompleteGuide, Author: JohnDoe
ISBN: 9012, Title: DataStructures, Author: AliceSmith
ISBN: 5678, Title: PythonBasics, Author: JaneDoe

Answer

```
import java.util.*;

// You are using Java
class Book {
    int isbn;
    String title;
    String author;

    Book(int isbn, String title, String author) {
        this.isbn = isbn;
        this.title = title;
        this.author = author;
    }

    // Uniqueness based on ISBN
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Book)) return false;
        Book b = (Book) obj;
        return this.isbn == b.isbn;
    }

    @Override
    public int hashCode() {
        return Integer.hashCode(isbn);
    }
}

class Library {
    LinkedHashSet<Book> set = new LinkedHashSet<>();
    // Add book
```

```

void addBook(int isbn, String title, String author) {
    set.add(new Book(isbn, title, author));
}

// Remove book by ISBN
void removeBook(int isbn) {
    Book toRemove = null;

    for (Book b : set) {
        if (b.isbn == isbn) {
            toRemove = b;
            break;
        }
    }

    if (toRemove != null) {
        set.remove(toRemove);
    }
}

// Display books
void displayBooks() {
    if (set.isEmpty()) {
        System.out.println("No books available");
        return;
    }

    for (Book b : set) {
        System.out.println("ISBN: " + b.isbn + ", Title: " + b.title + ", Author: " +
b.author);
    }
}

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Library library = new Library();
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            int isbn = sc.nextInt();
            String title = sc.next();
            String author = sc.next();

```

```
        library.addBook(isbn, title, author);
    }
    int m = sc.nextInt();
    for (int i = 0; i < m; i++) {
        int isbn = sc.nextInt();
        library.removeBook(isbn);
    }
    library.displayBooks();
    sc.close();
}
}
```

Status : Correct

Marks : 10/10